

Simulated annealing search

Representations give you a packing

The idea

- Pick a representation you already trust: slicing, B^* , or sequence pair
- Define moves that stay in that encoding
- At temperature T , accept worse costs with probability related to $\exp$ of minus delta over T
- Cool on a schedule
- Always re-pack and re-score after a move
- Illegal overflow should cost heavily so the search prefers legal packings

Browser lab track

- Open **simulated-annealing-fp**
- Run a short anneal on the starter instance
- Watch cost and temperature
- Pause, inspect the current packing, then continue
- Try a hotter start versus a greedy low temperature
- Then implement a tiny SA loop yourself with a fixed seed

Implement track

- Wire SA around one representation
- Log temperature, cost, and best packing
- Use a deterministic RNG seed
- Stop after a modest iteration budget on the toy outline
- Export the best coordinates and deadspace

Pitfalls

- Moves that don't change the packing waste iterations
- A cost that ignores overflow will “optimize” illegal layouts
- Cooling too fast freezes a bad packing
- Also don't compare costs across different outline penalties without documenting the

Your turn

- Get SA to improve deadspace or wirelength proxy from a random start while staying legal
- Next: soft modules, resize aspect under area constraints